Lab 5 : Scoping a Grounded Research Gap in LLM4APR

Topic : Beyond Accuracy@k: How Localization Quality Shapes Repository-Level LLM Program Repair

Juan (东万) – L25020002

I. BACKGROUND & CURRENT AI CAPABILITIES

Currently, Large Language Model (LLM)-based Automated Program Repair (APR) is primarily evaluated by tasking AI with fixing real-world GitHub issues in software projects, typically using the SWE-bench benchmark [3]. Research indicates that the most challenging step occurs before the AI even begins drafting a fix: it must first identify the location of the bug. The MAGIS study [2] found that pinpointing the exact line of code is a critical factor in resolving GitHub issues, noting that issues become increasingly difficult to resolve as the number of files and functions requiring modification grows. A recent review of LLM4APR [1] also emphasizes that bug localization is a prerequisite for generating a fix. This step is also costly; the SHERLOC study [7] reports that repair agents often exhaust nearly half of their allocated budget just on bug localization before any code editing begins. Consequently, this research focuses on localization the initial hurdle identified by MAGIS as an unresolved challenge worthy of further investigation.

In response, contemporary AI tools treat bug localization as a distinct task. Agentless [10] employs a hierarchical search narrowing down from files to functions, and finally to specific lines of code. Other approaches, such as LogAgent [4], CoSIL [5], and SweRank [6], utilize code graphs, call-graph analysis, and “search-then-rank” methodologies. SHERLOC [7] goes a step further by providing a brief explanation of the bug, rather than just its location. These tools have demonstrably improved repair outcomes; for instance, integrating CoSIL with Agentless increased success rates by 2.98%–30.5% [5]. Loc2Repair [8] represents the first study to evaluate localization and repair as separate processes under consistent conditions. Its findings on the “SWE-bench Verified” dataset show that using predicted file locations improves the success rate from 44.7% to approximately 49%, while utilizing the “gold” (perfectly accurate) file locations boosts the success rate to 52.4%.

II. THE UNRESOLVED GAP : FILE LOCALIZATION

In large-scale projects, localization tools can rarely determine with absolute certainty which file contains a bug. Consequently, rather than providing the AI with a single file, these tools supply a list of candidate files those most likely to be the source of the issue. The challenge lies in defining what constitutes an “optimal” list. If the list is too long, it will contain many files unrelated to the bug, potentially exceeding the AI's context window. Conversely, if the list is too short, the file actually containing the bug might be omitted, leaving the AI with no opportunity to fix it [14]. In other words, there is a trade-off between a precise list (high precision; fewer files, with most being relevant) and a comprehensive list (high recall; including all relevant files, but also incorporating irrelevant ones). Including more files is not always better. In the SWE-bench study [3], as analyzed by MAGIS [2], increasing the number of files examined improved the recall the proportion of actual bug-containing files identified from 29.58% to 51.06%; however, the success rate for Claude-2 actually declined from 1.96% to 1.22%.

III. ROOT CAUSE ANALYSIS

This gap persists due to three limitations in how localization has been studied to date:

1. **Overly simplistic testing setups**: Research that most directly tests this assumption operates solely at the function level, focusing on Java bugs that involve only a single function and comparing "perfect" (ground-truth) locations with those identified by automated tools. Repair Agent [11] explicitly assumes that bug locations are always perfectly accurate, and a recent survey found that 10 out of 66 LLM-based repair systems make the same assumption [14]. DAC [12] also assumes that the function containing the bug is already known, finding that allowing an AI to repair the entire function yields better results than first attempting to pinpoint the exact code fragment (hunk) to be fixed. Campos et al. [13] discovered that replacing perfect locations with the top three functions identified by automated tools reduces accuracy by 7.83–16.20% on a subset of the Defects4J dataset. This indicates that localization is crucial, yet it offers little insight into large-scale projects where a single fix may span multiple files.
2. **Use of search scores as a proxy for repair outcomes**: Localization tools are evaluated using search metrics such as Acc@k and Recall@k, with their utility for the repair process reflected only as an overall (end-to-end) improvement. Research on complete projects has merely compared a few fixed configurations: no

localization, predicted localization, and perfect localization [8]. Consequently, it remains unclear whether a tool should prioritize getting the first file correct (Acc@1) or ensuring that a greater number of relevant files appear within the top five (Recall@5). Furthermore, repairs involving multiple files are more challenging [2], and some issues remain unresolved even when perfect localization is provided [8]. Yet, none of the studies we reviewed have examined whether high-quality localization has a greater impact on such multi-file repairs.

3. **Passing tests equated with success**: All the results mentioned above consider a fix successful if it passes a benchmark test. However, when the testing criteria are made more rigorous, 19.71% of the fixes previously deemed successful produced by the top 30 agents on SWE-bench Verified are actually rejected [9]. The LLM4APR research agenda [1] also calls for an evaluation that accounts for costs and simultaneously examines both the localization and the verification of fixes; however, this has not yet been carried out for complete, end-to-end projects.

IV. PROPOSED SOLUTION

To address this gap, this study proposes a controlled experiment using SWE-bench Verified. The AI model and agent configuration remain constant, with the only variable being the list of files provided to the agent. In some experimental conditions, irrelevant files are added to the set of relevant files, while in others, some of the relevant files are removed. Bugs are also categorized based on the complexity of their fixes, following the analysis by MAGIS [2]. Furthermore, the study compares outcomes when the AI is provided only with the location of a bug versus when it is provided with both the location and a brief explanation, as in SHERLOC [7]. Additionally, the study evaluates whether the fixes are truly correct by employing more rigorous testing [9], rather than relying solely on whether they pass the initial test suite. Unlike Loc2Repair [8] which compares fixed configurations and previous function-level studies [11–13], this design incrementally adjusts precision and recall within the context of entire software projects. This approach is practical to implement, as it utilizes public benchmarks, open-weight models (such as Qwen2.5-Coder-32B, as used in [5]), and a curated subset of bugs to keep computational costs manageable.

V. FORMULATED RESEARCH QUESTIONS (RQS)

RQ1. Precision versus recall

How do the precision and recall of the list of files provided to an AI repair agent affect the repair success rate and the associated costs?

Independent Variables: Precision for file lists {1.0; 0.5; 0.25} (by adding irrelevant files) × recall {1.0; 0.5} (by retaining only half of the relevant files, rounded up). Recall is only varied for bugs involving multiple files, as single-file bugs can only have a recall of 0 or 1. The AI model, agent configuration, prompt, and set of bugs remain constant.

Dependent Variables: Repair success rate (%) and token cost per bug

RQ2. Interaction with change complexity

Does the impact of localization quality on repair success depend on the complexity of the required fix?

Independent Variables: Localization settings {no location, predicted location (e.g., CoSIL [5]), perfect location}. Moderator: complexity of the correct fix (single file vs. multiple files, number of code blocks modified)

Dependent Variable: Success rates were analyzed using logistic regression, which examined how localization settings and repair complexity influence one another.

RQ3. Ground truth accuracy

Does providing the AI with a brief explanation of the bug, in addition to its location, lead to a higher number of correct repairs compared to providing only the location?

Independent Variable: Types of localization output {file path; file+function; location accompanied by an explanation, as in SHERLOC[7]}.

Dependent Variables: Success rate on initial tests, correctness rate on more stringent tests [9] or manual inspection on samples, overfitting rate (improvements that pass initial tests but fail more stringent tests).

VI. REFERENCES

[1] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, Z. Chen. A Systematic Literature Review on Large Language Models for Automated Program Repair. ACM TOSEM, 2026. arXiv:2405.01466v4.

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. NeurIPS, 2024. arXiv:2403.17927.

[3] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024.

[4] Z. Chen et al. LocAgent: Graph-Guided LLM Agents for Code Localization. ACL, 2025. arXiv:2503.09089.

[5] Z. Jiang, X. Ren, M. Yan, W. Jiang, Y. Li, Z. Liu. Issue Localization via LLM-Driven Iterative Code Graph Searching (CoSIL). ASE, 2025. arXiv:2503.22424.

[6] R. G. Reddy, T. Suresh, J. Doo, et al. SweRank: Software Issue Localization with Code Ranking. ICLR, 2026. arXiv:2505.07849.

[7] H. Tamoyan, S. Narenthiran, E. Arakelyan, M. Mezini, B. Ginsburg. SHERLOC: Structured Diagnostic Localization for Code Repair Agents. arXiv:2606.24820, 2026.

[8] M. N. Al Awad, S. Ivanov. Loc2Repair: A Framework for Evaluating the Impact of File-Level Issue Localization in Repo-Level LLM Repair. GeCoIn Workshop @ IJCAI-ECAI, 2026. arXiv:2606.30963.

[9] B. Yu et al. SWE-ABS: Adversarial Benchmark Strengthening Exposes Inflated Success Rates on Test-based Benchmark. arXiv:2603.00520, 2026.

[10] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024.

[11] I. Bouzenia, P. Devanbu, M. Pradel. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. arXiv:2403.17134, 2024.

[12] J. Xu, Y. Fu, S. H. Tan, P. He. Aligning the Objective of LLM-based Program Repair. ICSE, 2025. arXiv:2404.08877.

[13] V. Campos, R. Shariffdeen, A. Ulges, Y. Noller. Exploring Generalizable Automated Program Repair with Large Language Models. arXiv:2506.03283, 2025.

[14] B. Yang, Z. Cai, F. Liu, B. Le, L. Zhang, T. F. Bissyandé, Y. Liu, H. Tian. A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications. arXiv:2506.23749, 2025.